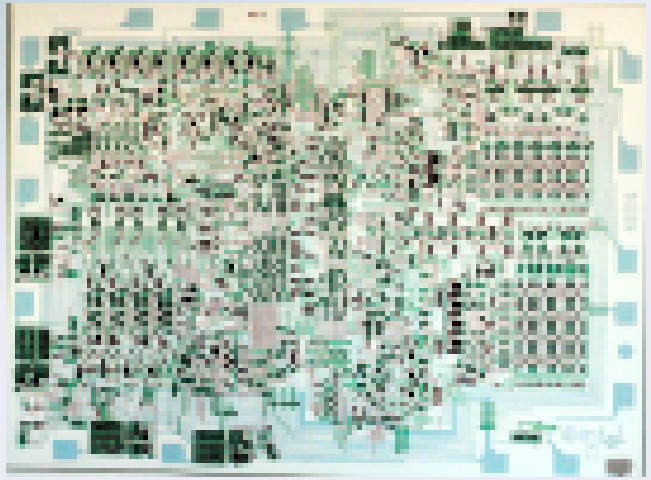

Lecture 5b: Programming: the LC-3

CSCI11: **Computer Architecture and Organization**

Readings

- *Chapter 7* Patt and Patel: Introduction to Computing Systems...
- *Guide to Using LC-3 Tools* - *Instructor Repo*
- *Appendix A* Patt and Patel: Introduction to Computing Systems... - *Instructor Repo*

5 Tabs Sidebar Back/Forward Privacy Page Tweaks Website Settings https://archive.org/details/introduction Bookmark New Tab 1Password Share Tab Overview Downloads Kagi Privacy Pass Overflow Menu
Saved Tabs Safari bookmarks Imported 3/28/25
MPC Lobo Apps Cloud Dashboard | Cle... LC3 - Google Drive Sebastian Rueda-villamil (Computer Ar... CSCI11_Sp26_Instructor/Week_5 at m... Introduction To Computing Systems: F...
INTERNET ARCHIVE WEB TEXTS VIDEO AUDIO SOFTWARE IMAGES SIGN UP | LOG IN UPLOAD Search
ABOUT BLOG EVENTS PROJECTS HELP DONATE CONTACT JOBS VOLUNTEER

CHAPTER

7 Assembly Language

By now, you are probably a little tired of 1s and 0s and keeping track of 0001 meaning ADD and 1001 meaning NOT. Also, wouldn't it be nice if we could refer to a memory location by some meaningful symbolic name instead of memorizing its 16-bit address? And wouldn't it be nice if we could represent each instruction in some more easily comprehensible way, instead of having to keep track of which bit of an instruction conveys which individual piece of information about that instruction? It turns out that help is on the way.

In this chapter, we introduce assembly language, a mechanism that does all of the above, and more.

7.1 Assembly Language Programming—Moving Up a Level

Recall the levels of transformation identified in Figure 1.9 of Chapter 1. Algorithms are transformed into programs described in some mechanical language. This mechanical language can be, as it is in Chapter 5, the machine language of a particular computer. Recall that a program is in a computer's machine language if every instruction in the program is from the ISA of that computer.

On the other hand, the mechanical language can be more user-friendly. We generally partition mechanical languages into two classes, high-level and low-level. Of the two, high-level languages are much more user-friendly. Examples are C, C++, Java, Fortran, COBOL, Python, plus more than a thousand others. Instructions in a high-level language almost (but not quite) resemble statements in a natural language such as English. High-level languages tend to be ISA independent. That is, once you learn how to program in C (or Fortran or Python) for one ISA, it is a small step to write programs in C (or Fortran or Python) for another ISA.

Before a program written in a high-level language can be executed, it must be translated into a program in the ISA of the computer on which it is expected to execute. It is often the case that each statement in the high-level language specifies several instructions in the ISA of the computer. In Chapter 11, we will introduce the high-level language C, and in Chapters 12 through 19, we will show the relationship between various statements in C and their corresponding translations to LC-3 code. In this chapter, however, we will only move up a small step from the ISA we dealt with in Chapter 5.

A small step up from the ISA of a machine is that ISA's assembly language. Assembly language is a low-level language. There is no confusing an instruction in a low-level language with a statement in English. Each assembly language instruction usually specifies a single instruction in the ISA. Unlike high-level languages, which are usually ISA independent, low-level languages are very much ISA dependent. In fact, it is usually the case that each ISA has only one assembly language.

The purpose of assembly language is to make the programming process more user-friendly than programming in machine language (i.e., in the ISA of the computer with which we are dealing), while still providing the programmer with detailed control over the instructions that the computer can execute. So, for example, while still retaining control over the detailed instructions the computer is to carry out, we are freed from having to remember what opcode is 0001 and what opcode is 1001, or what is being stored in memory location 0011111100001010 and what is being stored in location 0011111100000101. Assembly languages let us use mnemonic devices for opcodes, such as ADD for 0001 and NOT for 1001, and they let us give meaningful symbolic names to memory locations, such as SUM or PRODUCT, rather than use the memory locations' 16-bit addresses. This makes it easier to differentiate which memory location is keeping track of a SUM and which memory location is keeping track of a PRODUCT. We call these names *symbolic addresses*.

We will see, starting in Chapter 11, that when we take the larger step of moving up to a higher-level language (such as C), programming will be even more user-friendly, but in doing so, we will relinquish some control over exactly which detailed ISA instructions are to be carried out to accomplish the work specified by a high-level language statement.

7.2 An Assembly Language Program

We will begin our study of the LC-3 assembly language by means of an example. The program in Figure 7.1 multiplies the integer initially stored in NUMBER by 6 by adding the integer to itself six times. For example, if the integer is 123, the program computes the product by adding 123 + 123 + 123 + 123 + 123 + 123. Where have you seen that before? :-)

The program consists of 21 lines of code. We have added a *line number* to each line of the program in order to be able to refer to individual lines easily. This is a common practice. These line numbers are not part of the program. Ten lines start with a semicolon, designating that they are strictly for the benefit of

Flip left
Page 231 (260/801)
Introduction To Computing Systems: From Bits & Gates To C/C++ & Beyond (3rd Edition)
by Yale Patt, Sanjay M. Patel
Favorite Share Flag

appendix a

The LC-3 ISA

A.1 Overview

The Instruction Set Architecture (ISA) of the LC-3 is defined as follows:

Memory address space 16 bits, corresponding to 2^{16} locations, each containing one word (16 bits). Addresses are numbered from 0 (i.e, x0000) to 65,535 (i.e., xFFFF). Addresses are used to identify memory locations and memory-mapped I/O device registers. Certain regions of memory are reserved for special uses, as described in Figure A.1.

Bit numbering Bits of all quantities are numbered, from right to left, starting with bit 0. The leftmost bit of the contents of a memory location is bit 15.

Instructions Instructions are 16 bits wide. Bits [15:12] specify the opcode (operation to be performed), bits [11:0] provide further information that is

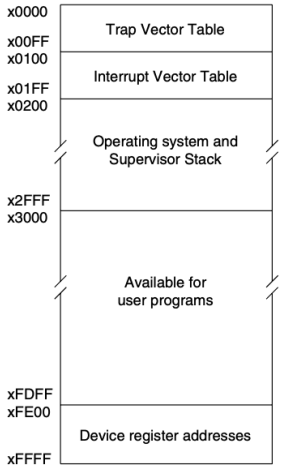

Figure A.1 Memory map of the LC-3

RTFM - Read the Fine Manual

- Single source of truth - Patt and Patel: Introduction to Computing Systems...
- Summary of all things LC3 - Appendix A (*Instructor Repo*) 26 pages

BTW

- ATmega328P datasheet - 653 pages
- Raspberry P Pico2 datasheet - 639 pages

LC-3 Assembly Language Syntax

- Each line of a program can be one of the following:
  - an *instruction* (1 of the 15 LC-3 *opcodes* plus *operands*)
  - an assembler *directive* (or *pseudo-op*)
  - a *comment*
- Whitespace (between symbols) and case are ignored, however, **very important**
- Comments (beginning with ";") are also ignored, **even more important**.

Instructions

- An *instruction* has the following format:
  LABEL **OPCODE OPERANDS** *; COMMENTS*
  (*optional* **mandatory**)
- *Instructions* are executed by the computer
- Are written in **assembly language**

Opcodes and Operands

- *Opcodes*
  - reserved symbols that correspond to **(LC-3) instructions**
  - ex: *ADD*, *AND*, *LD*, *LDR*
- *Operands*
  - *registers* -- specified by *Rn*, where *n* (0-7) is the register number
  - values (numbers) -- indicated by # (*decimal*) or x (*hex*)
  - *label* -- symbolic name of memory location
  - separated by commas
  - number, order, and type correspond to instruction format
  - details see **Appendix A** (*Instructor repo*)

Labels

- placed at the beginning of the line
- assigns a symbolic name to the address corresponding to line
- in this case, *DELAY* and *BIGNUM*

```
; delay loop for 32,767 clock cycles
        .ORIG x3000
        LD R1, BIGNUM
DELAY   ADD R1, R1, #-1
        BRp DELAY
        HALT
BIGNUM  .FILL   x7FFF
        .END
```

Comments

- anything after a semicolon is a comment
- ignored by assembler
- used by humans to document/understand programs
- tips for useful comments:
  - avoid restating the obvious, as "decrement R1"
  - provide additional insight, as in "*increment COUNT in R0*"
  - use comments to separate pieces of program

Comment Errors

NOTE: If there is a unidentifiable word on a line without a register reference, it appears to be assumed to be a LABEL

```
    OUS ; seen as a LABEL
    OUS R2 ; generates an error
```

Assembler Directives

- Pseudo-operations...o make programmer's life easier
- Do not refer to operations executed by program
- Used by *assembler*, similar to macros in *C*
- Look like *opcode*, however, *directive* starts with dot

Opcode	Operand	Meaning
.ORIG	address	starting address of program, **required**
.END		end of program, **required**
.BLKW	n	allocate n words of storage, *variable* or *array*
.FILL	n	allocate one word, initialize with value n, *constant*
.STRINGZ	n-character string	allocate n+1 locations, initialize w/characters and null terminator

Appendix A: The LC3 ISA Walkthrough

Table A.1 — Notational Conventions

Notation	Meaning
xNumber	The number in hexadecimal notation.
#Number	The number in decimal notation.
imm5	A 5-bit immediate value; bits [4:0] Range: −16..15.
offset6	A 6-bit value; bits [5:0] Range: −32..31.
LABEL	identifies a location symbolically
PC	Program Counter, memory address of the next instruction
PCoffset9	A 9-bit value; bits [8:0] of an instruction Range: −256..255.
PCoffset11	An 11-bit value; bits [10:0] of an instruction Range: −1024..1023.
mem[address]	Denotes the contents of memory at the given address.

Table A.1 — Notational Conventions (continued)

Notation	Meaning
setcc()	Condition codes N, Z, and P, set based on the **value of the result written to DR**
BaseR	Base Register; one of R0..R7
DR	Destination Register; one of R0..R7
SR, SR1, SR2	Source Register; one of R0..R7
trapvect8	An 8-bit value, used with the TRAP opcode to determine address of a trap service routine.
USP	The User Stack Pointer.
SEXT(A)	Sign-extend A.
ZEXT(A)	Zero-extend A.

ADD — Addition

Assembler Formats

```
ADD DR, SR1, SR2
ADD DR, SR1, imm5
```

Examples

```
ADD R2, R3, R4   ; R2 ← R3 + R4
ADD R2, R3, #7   ; R2 ← R3 + 7
```

AND — Bit-wise Logical AND

Assembler Formats

```
AND DR, SR1, SR2
AND DR, SR1, imm5
```

Examples

```
AND R2, R3, R4   ; R2 ← R3 AND R4
AND R2, R3, #7   ; R2 ← R3 AND 7
```

BR — Conditional Branch

Assembler Formats

```
BRn   LABEL      BRzp  LABEL
BRz   LABEL      BRnp  LABEL
BRp   LABEL      BRnz  LABEL
BR    LABEL      BRnzp LABEL
```

Examples

```
BRzp LOOP   ; Branch to LOOP if the last result was zero or positive.
BR   NEXT   ; Unconditionally branch to NEXT.
```

> **Note:** The assembly language opcode `BR` is interpreted the same as `BRnzp`; that is, always branch to the target address.

JMP / RET — Jump / Return from Subroutine

Assembler Formats

```
JMP BaseR
RET
```

Examples

```
JMP R2   ; PC ← R2
RET      ; PC ← R7
```

JSR / JSRR — Jump to Subroutine

Assembler Formats

```
JSR  LABEL
JSRR BaseR
```

Examples

```
JSR  QUEUE   ; Put the address of the instruction following JSR into R7;
             ; Jump to QUEUE.
JSRR R3      ; Put the address following JSRR into R7; Jump to the
             ; address contained in R3.
```

LD — Load

Assembler Format

```
LD DR, LABEL
```

Example

```
LD R4, VALUE   ; R4 ← mem[VALUE]
```

LDI — Load Indirect

Assembler Format

```
LDI DR, LABEL
```

Example

```
LDI R4, ONEMORE   ; R4 ← mem[mem[ONEMORE]]
```

LDR — Load Base+offset

Assembler Format

```
LDR DR, BaseR, offset6
```

Example

```
LDR R4, R2, #-5   ; R4 ← mem[R2 - 5]
```

LEA — Load Effective Address

Assembler Format

```
LEA DR, LABEL
```

Example

```
LEA R4, TARGET   ; R4 ← address of TARGET
```

Demo of load.asm

Comparing the results of the different load commands.

NOT — Bit-Wise Complement

Assembler Format

```
NOT DR, SR
```

Example

```
NOT R4, R2   ; R4 ← NOT(R2)
```

RTI — Return from Interrupt

Assembler Format

```
RTI
```

Example

```
RTI   ; PC, PSR ← top two values popped off stack
```

ST — Store

Assembler Format

```
ST SR, LABEL
```

Example

```
ST R4, HERE   ; mem[HERE] ← R4
```

STI — Store Indirect

Assembler Format

```
STI SR, LABEL
```

Example

```
STI R4, NOT_HERE   ; mem[mem[NOT_HERE]] ← R4
```

STR — Store Base+offset

Assembler Format

```
STR SR, BaseR, offset6
```

Example

```
STR R4, R2, #5   ; mem[R2 + 5] ← R4
```

TRAP — System Call

Assembler Format

```
TRAP trapvector8
```

Description

First R7 is loaded with the incremented PC. (This enables a return to the instruction physically following the TRAP instruction in the original program after the service routine has completed execution.) Then the PC is loaded with the starting address of the system call specified by trapvector8. The starting address is contained in the memory location whose address is obtained by zero-extending trapvector8 to 16 bits.

Example TRAP — System Call

```
TRAP x23   ; Directs the operating system to execute the IN system call.
           ; The starting address of this system call is contained in
           ; memory location x0023.
```

> **Note:** Memory locations x0000 through x00FF, 256 in all, are available to contain starting addresses for system calls specified by their corresponding trap vectors. This region of memory is called the Trap Vector Table.

Trap Codes

The assembler offers *"pseudo-instructions"*, for each specific *TRAP n* value

Code	Equivalent	Description
HALT	TRAP x25	Halt execution and print message to console, **required**
IN	TRAP x23	Print prompt on console, read (and echo) one character from keybd. Character stored in R0[7:0].
OUT	TRAP x21	Write one character (in R0[7:0]) to console.
GETC	TRAP x20	Read one character from keyboard. Character stored in R0[7:0].
PUTS	TRAP x22	Write null-terminated string to console. Address of string is in R0.

Demo of simple.asm

Recreate long delay program from above

Editor

1. Add BIGNUM as data, x7FFF
2. Use R1 to count
3. Continue to 0

Simulator

1. Show single-stepping
2. Review register 1 changing
3. Show how to reload and execute

simple.asm

```
; simple comment to describe code
        .ORIG x3000
        code, replace this line
        HALT
        data (optional), replace this line
        .END
```